

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Gnosis Safe Guard	Documentation quality	High	
Timeline	2025-11-18 through 2025-11-21	Test quality	High	
Language	Solidity	Total Findings	6	Fixed: 6
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review	High severity findings ⓘ	0	
Specification	None	Medium severity findings ⓘ	2	Fixed: 2
Source Code	https://github.com/Hypernative-Labs/hypernative-guard ⓘ #710f271 ⓘ	Low severity findings ⓘ	4	Fixed: 4
Auditors	- Hamed Mohammadi Auditing Engineer - Ibrahim Abouzied Auditing Engineer - Rabib Islam Senior Auditing Engineer	Undetermined severity findings ⓘ	0	
		Informational findings ⓘ	0	

Summary of Findings

The **HypernativeGuard** contract implements a Gnosis Safe smart account guard, enabling Hypernative and their clients to add additional validation logic to transactions executed by the Gnosis Safe by implementing the `ITransactionGuard.checkTransaction()` function.

The contract supports two flows for transaction verification:

- Keeper signature verification** – Transactions are signed off-chain by the keeper and verified on-chain.
 - Whitelist verification** – Hashes of specific allowed transactions can be added to a whitelist for automatic approval.
- Additionally, Safe users can introduce extra policies to verify transactions through the `addPolicyExtension()` function.

The guard also allows for temporarily bypassing transaction verification by setting the `isPassThroughMode` flag, as well as permanently removing the guard through a revocation flow that is protected by a time lock.

During the audit, we identified several issues regarding the operations of the vault and scenarios in which a malicious user could exploit the system to take over or cause a denial of service (DoS). In particular, the `isPassThroughMode` flag is very powerful: once enabled, it allows transactions to bypass the guard entirely, which could lead to guard revocation or a takeover by changing the keeper.

Fix-Review Update 2025-12-09:

Repository: `https://github.com/Hypernative-Labs/hypernative-guard` Commit: `ab3fa31e8f3f0c03f2bf7e924b8259b45a6d9b5c`

All issues identified during the audit have been fixed, mitigated, or acknowledged, and the provided fixes were reviewed.

Fix-Review Update 2025-11-29:

Most of the issues identified during the audit have been fixed, mitigated, or acknowledged, with one remaining unresolved. Additionally, during the fix review we identified a new issue that was added to the report.

ID	DESCRIPTION	SEVERITY	STATUS
HYP-1	Keeper Signature Bypasses Timelock Safeguard for Guard Revocation	● Medium ⓘ	Fixed
HYP-2	Consider Using Function Selectors Instead of Full Safe Transaction Hashes to Do Catch All Instances of a Sensitive Function Call	● Medium ⓘ	Fixed

ID	DESCRIPTION	SEVERITY	STATUS
HYP-3	Keeper Can Unilaterally Bypass All Security Policies	• Low ⓘ	Fixed
HYP-4	<code>enablePassThroughMode()</code> Has No Time-Lock	• Low ⓘ	Fixed
HYP-5	A Guard with No Keeper Can Be Frozen by Broken Policies	• Low ⓘ	Fixed
HYP-6	<code>_extractSignatureComponents()</code> May Underflow or Return Incorrect Data	• Low ⓘ	Fixed

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

i

Disclaimer

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
 1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
 2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
 3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
 1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
 2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

Files Included

Repo: `https://github.com/Hypernative-Labs/hypernative-guard`

Included Paths: `src/`

Extensions: `sol`

Operational Considerations

DoS by Running Out of Gas

Gas consumption by policy extensions can cause a denial-of-service. The guard executes external calls to policy extensions within a loop. A malicious or poorly coded policy could consume excessive gas, causing any transaction routed through the guard to fail due to out-of-gas errors. This would effectively freeze all guarded transactions from the Safe.

Centralization of Power in Keeper Role – Single Point of Failure

The keeper role holds critical permissions to approve, revoke, and bypass security policies. If a keeper’s key is compromised, the entire security model of the guard fails. This could be mitigated by implementing a threshold system for the keeper role, whereby multiple keepers are required to execute a keeper-gated function.

Signature Padding Requirement

If a `keeperSignature` is not being used, the `signatures` parameter in `checkTransaction()` must be padded with an additional 97 empty bytes at the end to avoid a revert.

Guard Can Be Instantly Revoked in Pass Through Mode

A time-lock is enforced for removing the guard under normal operations. However, if `isPassThroughMode == true`, all transactions are approved and the time-lock is bypassed.

Guard Is Initialized in an Insecure Default State

The constructor sets `isPassThroughMode = true` by default. This means that upon deployment and being set as the guard, it provides no protection until the keeper explicitly calls `disablePassThroughMode()`.

Possibility of guard takeover due to keeper mistake

The two functions `grantKeeperRole()` and `revokeKeeperRole()` use the `onlyGuardedSafe` modifier. This means that if, by mistake, a keeper verifies a malicious call to these functions, a malicious actor can take over the guard.

Key Actors And Their Capabilities

Keeper

The `KEEPER_ROLE` is maintained by Hypernative and is in charge of signing valid transactions off-chain, maintaining the transaction hash whitelists, and enabling the `isPassThroughMode`.

The Safe

The Gnosis Safe is guarded by the Hypernative Guard. Every transaction sent to the Safe has to pass through the Guard. List of specific transactions within the Guard that can be initiated via the Safe:

- Disabling the `isPassThroughMode`
- Adding and removing new policies
- Activating and disabling the time lock
- Revoking the Guard

Findings

HYP-1

Keeper Signature Bypasses Timelock Safeguard for Guard Revocation

• Medium ⓘ Fixed



Update

Marked as "Fixed" by the client.
Addressed in: `1a6da3dac1f1a702d4f44359a0cd91e691cd80fc`.
The client provided the following explanation:

`moved the signature verification flow to a later step in the checkTransaction flow.`

Issue was fixed by checking the special pre-approved transaction hashes beforehand and making sure they follow the hardcoded process in the contract, such as adhering to the time-lock, before going through verification by the keeper.

File(s) affected: `src/HypernativeGuard.sol`

Description: The contract logic in `checkTransaction()` is structured to validate a keeper's signature when it is present instead of checking for pre-approved hashes and the timelock. A keeper can provide a signature for a guard-revocation transaction, allowing it to execute immediately and bypass the otherwise-mandatory time delay, whereas this is an important defense against takeover.

Recommendation: The validation logic should be refactored to ensure the timelock is always necessary for guard revocation.

HYP-2

Consider Using Function Selectors Instead of Full Safe Transaction Hashes to Do Catch All Instances of a Sensitive Function Call

• Medium ⓘ Fixed

✓ Update

The issue was fixed in `7507da3208ab2ee0ce120bfb1c15e667037c4eda` and `d628ae83fdb846ad9a6607bf262835e033e3c49a`.

File(s) affected: `src/HypernativeGuard.sol`

Description: The contract defines several special functions that can be executed through the Safe to manage the guard, such as `setGuard()`, `revokeKeeperRole()`, `enablePassThroughMode()`, `activateTimelock()`, and others. Currently, calls to these functions are identified by computing and storing a full transaction hash that includes the function selector, encoded parameters, and all Safe execution parameters (`operation`, `safeTxGas`, `gasPrice`, `refundReceiver`, etc.), each set to default values.

While these additional parameters are necessary for normal Safe-mediated calls—where the keeper must sign the full transaction hash—they introduce a weakness for these privileged special functions. A malicious actor can invoke a special function with slightly modified Safe parameters, causing the hash to differ from the one originally registered. If a keeper accidentally signs the modified hash, the attacker can perform an unauthorized privileged action, such as bypassing the timelock when revoking the guard.

Exploit Scenario:

1. **During deployment**, Hypernative computes and stores the revocation hash using default Safe parameters:

```
revokingHash = keccak256(abi.encode(
    to: address(safe),
    value: 0,
    data: abi.encodeWithSelector(GuardManager.setGuard.selector, address(0)),
    operation: Enum.Operation.Call,
    safeTxGas: 0,
    baseGas: 0,
    gasPrice: 0,
    gasToken: address(0),
    refundReceiver: payable(0),
    nonce: safe.nonce()
));
```

2. **An attacker later crafts a slightly modified version** of the same call, such as changing only the `refundReceiver` field:

```
revokingHash = keccak256(abi.encode(
    to: address(safe),
    value: 0,
    data: abi.encodeWithSelector(GuardManager.setGuard.selector, address(0)),
    operation: Enum.Operation.Call,
    safeTxGas: 0,
    baseGas: 0,
    gasPrice: 0,
    gasToken: address(0),
    refundReceiver: payable(safe),
    nonce: safe.nonce()
));
```

3. **If the keeper signs this modified hash**, the attacker can revoke the guard even though the hash does not match the one registered during deployment. This bypasses the timelock flow for guard revocation.

If `_validateTransactionApproval()` function instead of hash checked only the function selector, encoded parameters (and `to` address) for special privileged operations, the call would be rejected before signature validation. This prevents both accidental and malicious keeper signatures from enabling unauthorized guard modifications.

Recommendation: For regular arbitrary function calls, full-hash validation remains necessary. However, for privileged special functions, relying solely on the function selector, encoded parameters (combined with the `to` address) provides stricter control. Selector-level matching ensures that *any* invocation of a sensitive function is caught, regardless of Safe parameter variations, instead of relying on hash equality that requires every parameter to match exactly.

HYP-3 Keeper Can Unilaterally Bypass All Security Policies

• Low ⓘ Fixed

✓ Update

Marked as "Fixed" by the client.
Addressed in: 97e03635381769bea27dcef49e7b2a6210cca08c .
The client provided the following explanation:

```
changed so only the safe can enablePassThrough via onlyGuardedSafe modifier
```

File(s) affected: src/HypernativeGuard.sol

Description: A single keeper can call `enablePassThroughMode()` to disable all guard checks or `removePolicyExtension()` to remove a security policy added by the Safe's multi-sig owners. This gives one entity the power to nullify the collective security decisions of the Safe. The principle of least privilege is thereby violated.

Exploit Scenario:

1. Safe multi-sig signers add a policy extension.
2. Safe multi-sig signers sign a transaction. They expect the policy extension to be in effect because passthrough mode is disabled.
3. A keeper detects the transaction in a mempool and front-runs the transaction by either enabling passthrough mode or removing the policy extension.
4. The transaction bypasses the policy extension's validation without the Safe multi-sig signers' permission.

Recommendation: The power to change the security posture of the guard should belong either exclusively to the Safe owners, or the keeper should be beholden to a timelock.

HYP-4 `enablePassThroughMode()` Has No Time-Lock

• Low ⓘ

Fixed

✓ Update

Marked as "Fixed" by the client.
Addressed in: 97e03635381769bea27dcef49e7b2a6210cca08c .
The client provided the following explanation:

```
introduced timelock for enablePassThroughMode call
```

File(s) affected: src/HypernativeGuard.sol

Description: If `isPassThroughMode == true`, `checkTransaction()` immediately returns and the guard does no validation on the transaction. `isPassThroughMode` is set to `true` by default, and the safe can later call `disablePassThroughMode()` and begin performing transaction validation.

However, a `KEEPER_ROLE` holder can call `enablePassThroughMode()` to cease transaction validations. Additionally, `KEEPER_ROLE` can be granted by the safe via `grantKeeperRole()`, conditional on the function hash being whitelisted.

If a compromised safe is able to get ahold of a `KEEPER_ROLE` account, the attacker could bypass all transaction validations. While revoking the guard entirely is mitigated via a time-lock, `enablePassThroughMode()` would bypass transactions immediately.

Recommendation: A few possible solutions exist:

1. To prevent a compromised safe from assigning the `KEEPER_ROLE`, update `grantKeeperRole()` to use the modifier `onlyKeeper`.
2. Add a time-lock to `enablePassThroughMode()`.

HYP-5 A Guard with No Keeper Can Be Frozen by Broken Policies

• Low ⓘ

Fixed

✓ Update

Marked as "Fixed" by the client.
Addressed in: 6bd3bb80b7bf12f27f5ceec31e350ce2d1af709b .
The client provided the following explanation:

```
update to using AccessControlEnumerable in order verifying 1 keeper always exists.
```

The provided fix checks whether the contract has more than 0 keepers before revoking the existing keeper; however, this does not resolve the issue when exactly 1 keeper is assigned. You should either require more than 1 keeper to exist before allowing a revocation, or perform the check for more than 0 keepers after the revocation occurs.
Additionally, `AccessControl.renounceRole()` has not been overridden.

The issue was addressed in: 126102e74cdcdde44989d4495064a424e5de91f0 .

File(s) affected: src/HypernativeGuard.sol

Description: A safe can add an `IGuardPolicyExtension` that performs additional validation on each transaction. If a policy incorrectly reverts for any reason, a `KEEPER_ROLE` can call `removePolicyExtension()` and allow the safe to execute transactions once more.

However, the `KEEPER_ROLE` can be revoked by the safe is the `revokeKeeperRole()` function is approved. A Keeper-less contract cannot be rescued if it has a malignant or malfunctioning policy, and all funds may be frozen.

Recommendation: Update `revokeKeeperRole()` and `AccessControl.renounceRole()` to ensure that the guard always has at least one `KEEPER_ROLE`

HYP-6

• Low ⓘ Fixed

`_extractSignatureComponents()` May Underflow or Return Incorrect Data



Update

Marked as "Fixed" by the client.
Addressed in: `5b7741e58011dc0c7c5659fd4de04eeb41ada8d8` .
The client provided the following explanation:

`fixed potential underflow`

File(s) affected: `src/HypernativeGuard.sol`

Description: The `_extractSignatureComponents()` function assumes that the signatures `calldata` always ends with the suffix of the format "`[keeperSignature (65 bytes)][context][contextLength (32 bytes)]`". This is a departure from the traditional usage of `checkTransaction()` and an attempt to add another parameter to the function by overloading the `bytes signatures` array.

If the function is called with only raw signatures and without this structured trailer, then `contextLength` will be assigned as arbitrary data:

```
// Read context length from the last 32 bytes
uint256 contextLength = uint256(bytes32(signatures[signatures.length - 32:]));
```

If `contextLength` is larger than `signatures.length` , this will lead to an underflow in the line:

```
uint256 contextStartIndex = signatures.length - 32 - contextLength;
```

- Ultimately, the function will either:
1. Revert due to arithmetic underflow, or
 2. Succeed but return incorrect slices for both `keeperSignature` and `context` .

Recommendation: It is difficult to enforce a format for a `bytes` array of undetermined length. Consider wrapping the function in a `try / catch` statement. There should also be additional validation done on the `context` in the event that an underflow doesn't occur, but the `context` is *still* an invalid slice.

Auditor Suggestions

S1 Gas Optimization

Fixed



Update

Marked as "Fixed" by the client.
Addressed in: `873a91ccc044f25ccc6553bf1bcc87ffa9d3cc3b` .
The client provided the following explanation:

`Introduced suggested gas optimizations`

File(s) affected: `src/HypernativeGuard.sol`

Description: Gas usage can be optimized in a few places:

1. `extractSignatureComponents()` – cache `contextLengthStartIndex = signatures.length - 32` .
2. `checkTransaction()` – calculating `txHash` takes two calls: one to get the nonce, and another to get the `txHash` . Spend less gas by using an internal function to calculate the hash.

Recommendation: Minimize gas usage by implementing the above suggestions.

S2 Missing Input Validation

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: 589c9e30b1126fc0bd29f4c6d20b6bde0558e808 .

The client provided the following explanation:

added more input validations

File(s) affected: src/HypernativeGuard.sol

Related Issue(s): [SWC-123](#)

Description: It is important to validate inputs, even if they only come from trusted addresses, to avoid human error. Specifically, in the following functions arguments of type `address` may be initialized with value `0x0` :

1. `constructor() - _safeAddress , _keeper`
2. `grantKeeperRole() - _keeper`
3. `removeKeeperRole() - _keeper`

Recommendation: We recommend adding the relevant checks.

S3 Use Scoped Blocks or Group Variables with Structs

Fixed

✓ Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

unfortunately this recommendation doesn't meet our need, as the context may be extracted within the block but is unusable, as if the `checkPolicyExtensions` function will move to the block as well, the problem returns, and the context shall be used in the `checkPolicyExtensions` flow, appreciate another suggestion for this.

After careful review of the requirements, a sample implementation of this suggestion was provided to the client.

The suggestion was successfully applied in 5edbeab069cf27e16c6eff8ade7fd70d5d5bb992 .

File(s) affected: src/HypernativeGuard.sol

Description: If a `stack too deep` error is being encountered, consider grouping variables into structs, or introducing a new Scoped Block inside the function.

Recommendation: A few options include:

1. Packing the parameters of `checkTransaction()` into a `Transaction` struct, and calling a private `_checkTransaction(Transaction t)` function.
2. Update `_extractSignatureComponents()` to return a struct.
3. Update the `IGuardPolicyExtension` interface to take in a `Transaction` struct as a parameter.
4. Adding a new scoped block around `_extractSignatureComponents()` creates a temporary scope in which the `context` variable can be used without causing a "stack too deep" issue.

```
...
bytes memory keeperSignature;
{
    bytes memory context;
    (keeperSignature, context) = _extractSignatureComponents(signatures);
    //The context only exists within this block.
}
...
```

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

- Repo: `https://github.com/Hypernative-Labs/hypernative-guard`
- `4df...d10 ./src/HypernativeGuard.sol`
 - `ea7...557 ./src/IGuardPolicyExtension.sol`
 - `227...1a0 ./src/interfaces/ISafe.sol`

Test Suite Results

The test suite depends on deployed Safe instances and requires access to an owner's private key, which makes the testing process more complex. To address this and collect test results, we made targeted adjustments to the test setup. Additionally, running the tests after the fix review was also successful.

During the fix review, the test suite improved drastically, increasing from 18 original tests to 48 tests.

```
Ran 48 tests for test/HypernativeGuard.t.sol:HypernativeGuardTest
[PASS] testConfigureHypernativeGuard() (gas: 105524)
[PASS] testRevokeHypernativeGuardRevertsOnTimelockInit() (gas: 178333)
[PASS] testRevokeHypernativeGuardRevertsOnTimelockStillActive() (gas: 263113)
[PASS] test_ApproveAndRevokeNonceFree() (gas: 215822)
[PASS] test_AtLeastOneKeeperRequired() (gas: 179718)
[PASS] test_CannotRevokeTimelockHashes_ActivateHash() (gas: 11492)
[PASS] test_CannotRevokeTimelockHashes_DisableHash() (gas: 11505)
[PASS] test_ConfigureHypernativeGuardAndAbortUnapprovedTx() (gas: 153014)
[PASS] test_ConfigureHypernativeGuardAndExecuteWithdrawTx() (gas: 188400)
[PASS] test_ConstructorZeroAddressKeeper() (gas: 135670)
[PASS] test_ConstructorZeroAddressSafe() (gas: 155490)
[PASS] test_DisablePassThroughTimelock() (gas: 250290)
[PASS] test_EnablePassThroughMode() (gas: 222290)
[PASS] test_FunctionCallHash() (gas: 307605)
[PASS] test_GetPassThroughTimelockBlock() (gas: 130321)
[PASS] test_GetRevokeTimelockBlock() (gas: 202285)
[PASS] test_GrantAndRevokeKeeperRole() (gas: 377350)
[PASS] test_GrantKeeperRoleZeroAddress() (gas: 175485)
[PASS] test_NonceFreeTx() (gas: 292947)
[PASS] test_OnlyGuardedSafe_ActivatePassThroughTimelock() (gas: 8700)
[PASS] test_OnlyGuardedSafe_ActivateRevokeTimelock() (gas: 8676)
[PASS] test_OnlyGuardedSafe_AddPolicyExtension() (gas: 10915)
[PASS] test_OnlyGuardedSafe_DisablePassThroughTimelock() (gas: 8741)
[PASS] test_OnlyGuardedSafe_DisableRevokeTimelock() (gas: 8718)
```

```
[PASS] test_OnlyGuardedSafe_EnablePassThroughMode() (gas: 8698)
[PASS] test_OnlyGuardedSafe_GrantKeeperRole() (gas: 8852)
[PASS] test_OnlyGuardedSafe_RevokeKeeperRole() (gas: 10938)
[PASS] test_OnlyKeeperApproveFunctionCallHash() (gas: 10904)
[PASS] test_OnlyKeeperOrSafe_DisablePassThroughMode_AsKeeper() (gas: 14545)
[PASS] test_OnlyKeeperOrSafe_DisablePassThroughMode_Unauthorized() (gas: 11316)
[PASS] test_OnlyKeeperOrSafe_RemovePolicyExtension_Unauthorized() (gas: 13423)
[PASS] test_OnlyKeeperRevokeFunctionCallHash() (gas: 10895)
[PASS] test_OnlyKeeperRevokeHash() (gas: 10928)
[PASS] test_OnlyKeeperRevokeNonceFreeHash() (gas: 10952)
[PASS] test_PolicyExtension() (gas: 557622)
[PASS] test_PolicyExtensionAlreadyExists() (gas: 313923)
[PASS] test_PolicyExtensionNotFound() (gas: 182096)
[PASS] test_PolicyExtensionNotValid() (gas: 175532)
[PASS] test_RemovePolicyExtension_AsKeeper() (gas: 201014)
[PASS] test_RevokeApprovedHashExpectRevert() (gas: 165359)
[PASS] test_RevokeHypernativeGuardSuccessfully() (gas: 286003)
[PASS] test_RevokeKeeperRoleNotFound() (gas: 315605)
[PASS] test_SendFunds() (gas: 18063)
[PASS] test_SupportsInterface() (gas: 10157)
[PASS] test_TimelockCancelation() (gas: 249586)
[PASS] test_WithdrawEth() (gas: 88033)
[PASS] test_WithdrawEthOfflineSigner() (gas: 90768)
[PASS] test_addHashNotKeeperReverts() (gas: 10903)
Suite result: ok. 48 passed; 0 failed; 0 skipped; finished in 7.70s (28.76s CPU time)

Ran 1 test suite in 7.73s (7.70s CPU time): 48 tests passed, 0 failed, 0 skipped (48 total tests)
```

Changelog

- 2025-11-21 - Initial report
- 2025-12-05 - Final report
- 2025-12-10 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp’s mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 500 audits and secured over \$200 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Polygon, Avalanche, Curve, Solana, Compound, Lido, MakerDAO, Arbitrum, OpenSea and the World Economic Forum.

Quantstamp’s collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Near, Flow, Avalanche, Solana, Cardano, Binance Smart Chain, Hedera Hashgraph, Tezos
- DeFi: Curve, Compound, Maker, Lido, Polygon, Arbitrum, SushiSwap
- NFT: OpenSea, Parallel, Dapper Labs, Decentraland, Sandbox, Axie Infinity, Illuvium, NBA Top Shot, Zora
- Academic institutions: National University of Singapore, MIT

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

